Variables and the Memory Model

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

To find the area of a triangle with base 20 and height 12, we could use the following expression:

```
>>> 20 * 12 / 2
120.0
```

We could alternatively define a **variable**, which gives a value a name, so we can use it again.

```
>>> base = 20
>>> height = 12
>>> base * height / 2
120.0
```

using variables lets us change the values without retyping the whole formula

A variable name has to follow two **rules**:

(1) it starts with a letter or an underscore (_)

(2) it contains only letters, digits and underscores

allowed	**not allowed**	
`shoe_size`	`2nd_place`	starts with a digit
`_total`	`my-age`	contains -
`player2`	`top speed`	contains a space

Names are case-sensitive, so `seconds` and `Seconds` are different variables. By convention, `Python` names use **pothole_case**: lowercase words separated by underscores.

Which of these is a valid Python variable name?

A `2fast`

B `top-speed`

✓ `top_speed_2`

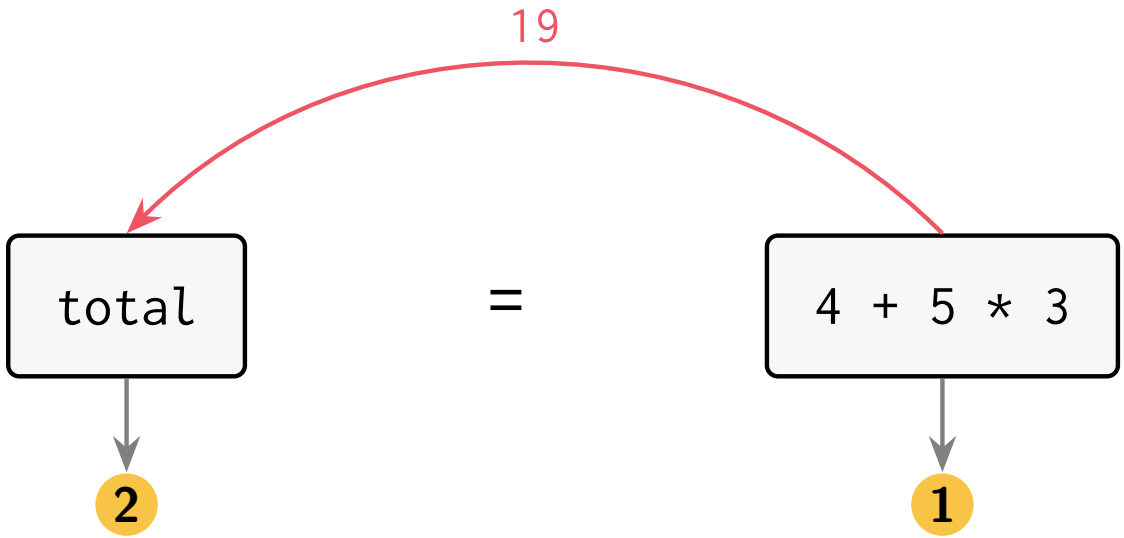
D `top speed`

Every value is stored in **memory** at a **memory address**. A variable holds the address of a value, so we say the variable **refers to** that value.

```
shoe_size = 8.5
age = 44
```

We say "`shoe_size` refers to `8.5`", not "`shoe_size` is `8.5`".

An **assignment statement** makes a variable refer to a value. Python runs it in two steps, right side first.



If the variable doesn't exist yet, Python creates it; otherwise it reuses it. Because of this order, `x = x + 1` is a perfectly good statement, even though it makes no sense as math.

grade.ipynb

A student earns `n` marks and `b` bonus marks on a test out of `total` possible marks.
Write a Python expression to produce the student's grade as a percentage between 0 and 100.
For example, if `n` refers to 7, `b` refers to 1, and `total` refers to 10, the expression should produce `80.0`.

In [1]

```
n = 7
b = 1
total = 10

(n + b) / total * 100
```

```
80.0
```

split_the_bill.ipynb

A group of friends goes to a restaurant. The total bill costs `total_cost` and it will be split evenly between `n` people.

Write a Python *expression* that produces the amount each person should pay.
(It is fine if the amount has more than two digits after the decimal place.)

In [1]

```
total_cost = 21
n = 4

total_cost / n
```

```
5.25
```

If `total_cost` refers to `21` and `n` refers to `4`, what value does `total_cost / n` produce, and what is its type?

```
1  total_cost = 21
2  n = 4
```

A `5`, an `int`

✓ `5.25`, a `float`

C `5.0`, a `float`

D `5.25`, an `int`

If `total_cost` refers to `20` and `n` refers to `4`, what value does `total_cost / n` produce, and what is its type?

```
1  total_cost = 20
2  n = 4
```

A `5`, an `int`

✓ `5.0`, a `float`

C `5.25`, a `float`

D `5`, a `float`

pizza.ipynb

Your roommate ordered pizzas to be shared among a group of people. They ordered `num_pizzas` pizzas each containing `num_slices` slices of pizza and there are `num_people` people.

(a) Write a Python *assignment statement* that assigns the number of whole slices each person gets to a variable named `slices_per_person`.

(b) Create a variable named `leftover` that refers to the number of slices left over.

In [ ]

```
num_pizzas = 2
num_slices = 10
num_people = 3

slices_per_person = num_pizzas * num_slices // num_people
leftover = num_pizzas * num_slices % num_people
```

In [2]

```
slices_per_person, leftover
```

```
(6, 2)
```

If `num_pizzas` refers to `2`, `num_slices` refers to `10` and `num_people` refers to `3`, what value does `slices_per_person` refer to?

```
num_pizzas = 2
num_slices = 10
num_people = 3
```

Answer: `6`

With the same values, what value does `leftover` refer to?

```
num_pizzas = 2
num_slices = 10
num_people = 3
```

Answer: `2`

What type are the values that `slices_per_person` and `leftover` refer to?

```
1  num_pizzas = 2
2  num_slices = 10
3  num_people = 3
```

- ✓ `int`
- B `float`

the memory model

A **memory model diagram** shows the state of memory after some code runs: variables on one side, values on the other.

```
a = 5
```

(1) each variable holds an **address**

(2) each value has its address, its type, and the value itself

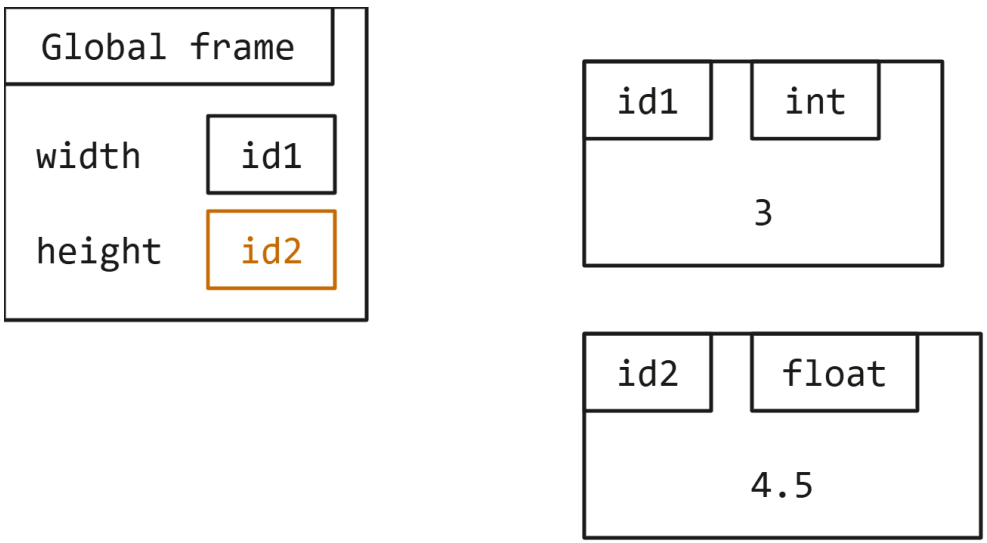
(3) a new value gets a new address: `id1`, `id2`, ...

memory_model.py

Program

```
1  width = 3
2  height = 4.5
```

Memory model (step 2)



The built-in function `id` gives a value's real memory address. It's a long number, which is why our diagrams use short labels instead.

```
>>> a = 5
>>> id(a)
4310763120
>>> b = a
>>> id(b)
4310763120
```

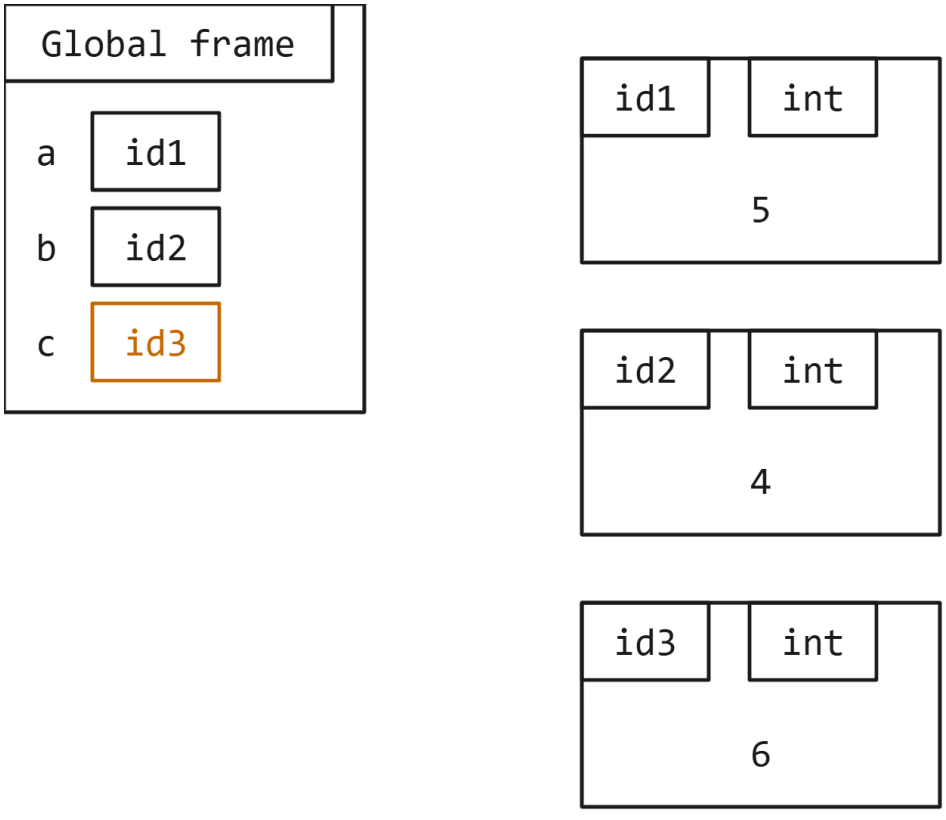
Both have the same address, so `a` and `b` refer to the very same value. Your numbers will be different.

memory1.py

Program

```
1  # Draw the memory model after this code runs.
2  a = 5
3  b = 4
4  c = 6
```

Memory model (step 3)

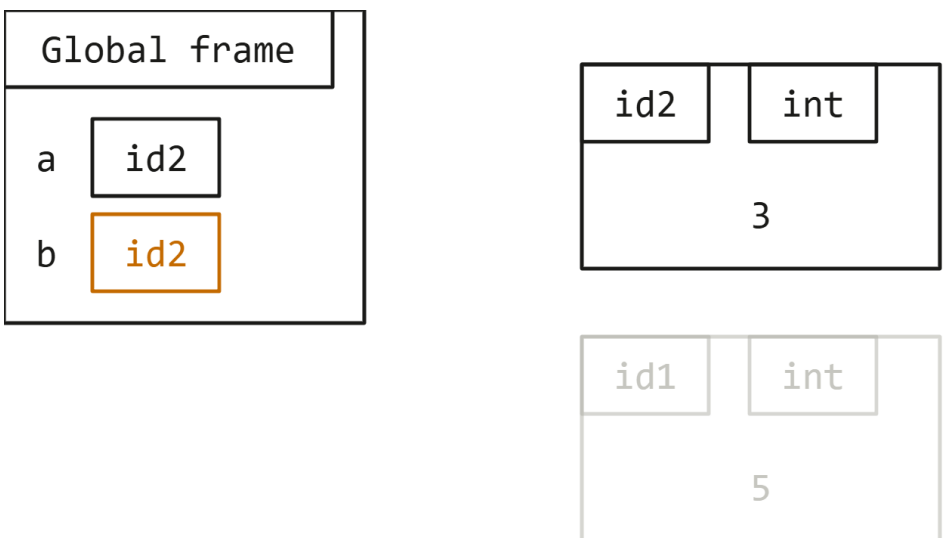

Global frame
a
id1
b
id2
c
id3
id1
int
5
id2
int
4
id3
int
6

memory2.py

Program

```
1  # Draw the memory model after this code runs.
2  a = 5
3  a = 3
4  b = a
```

Memory model (step 3)

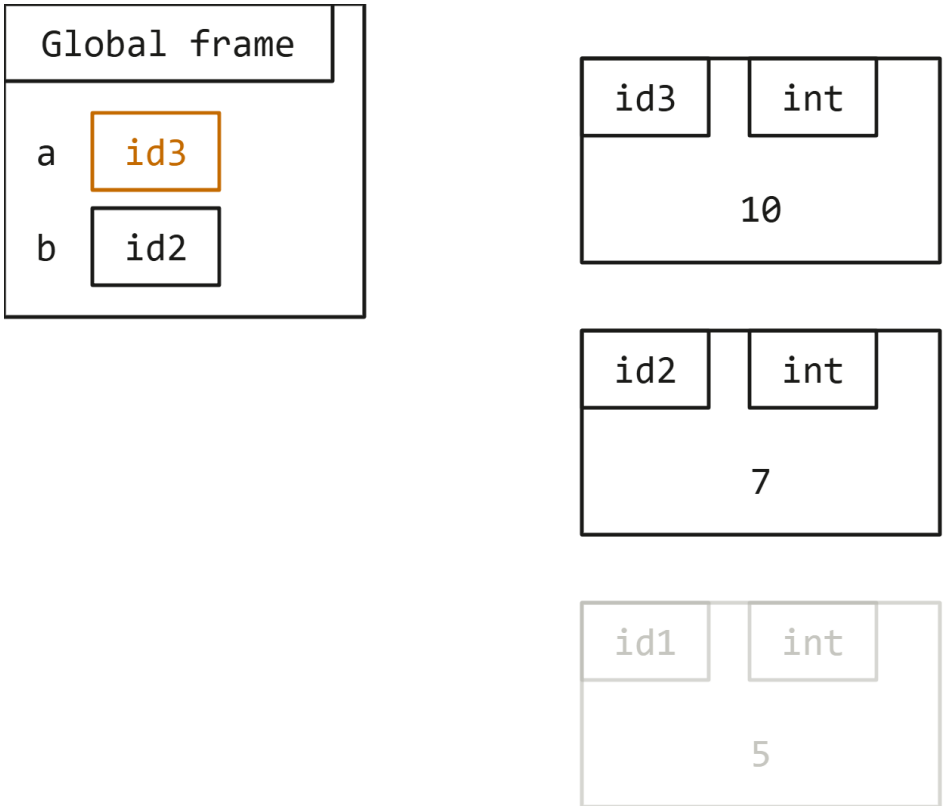


memory3.py

Program

```
1  # Draw the memory model after this code runs.
2  a = 5
3  b = a + 2
4  a = 10
```

Memory model (step 3)


Global frame
a
id3
b
id2
id3
int
10
id2
int
7
id1
int
5

reassigning variables

When a variable is assigned again, it gets a new address. Nothing else changes.

```
base = 20
height = 12
area = base * height / 2
base = 4
```

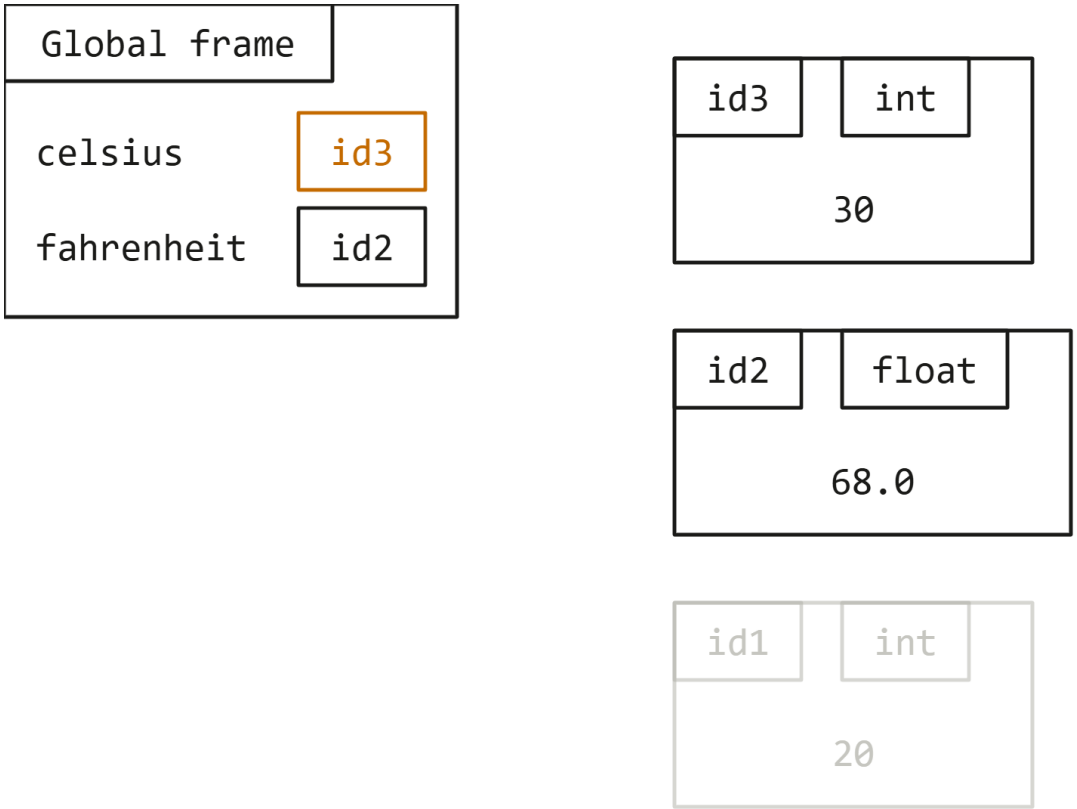
`area` still refers to `120.0`: it was computed once, from the values at that time.

reassign.py

Program

```
1  celsius = 20
2  fahrenheit = celsius * 9 / 5 + 32
3  celsius = 30
```

Memory model (step 3)


Global frame
celsius
id3
fahrenheit
id2
id3
int
30
id2
float
68.0
id1
int
20

Variables are **not tied** together. A variable on the right-hand side is replaced by its value, and that's the end of the connection.

```
x = 7
y = x * 6
x = 8
```

after	x	y
`x = 7`	7	—
`y = x * 6`	7	42
`x = 8`	8	42

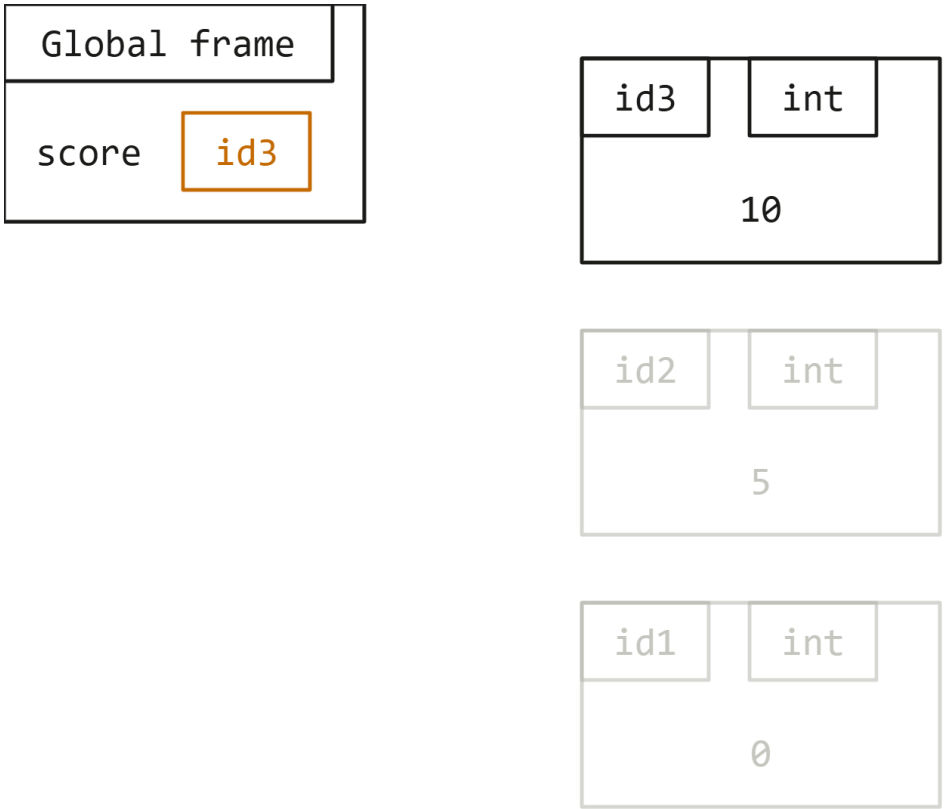
In the memory model, `x = 8` gives `x` a new address, and `y` still holds the address of `42`. The old value stays in memory until nothing refers to it.

score.py

Program

```
1  score = 0
2  score = score + 5
3  score = score + 5
```

Memory model (step 3)


Global frame
score
id3
id3
int
10
id2
int
5
id1
int
0

After this code runs, what value does `y` refer to?

```
1  x = 37
2  y = x + 2
3  x = 20
```

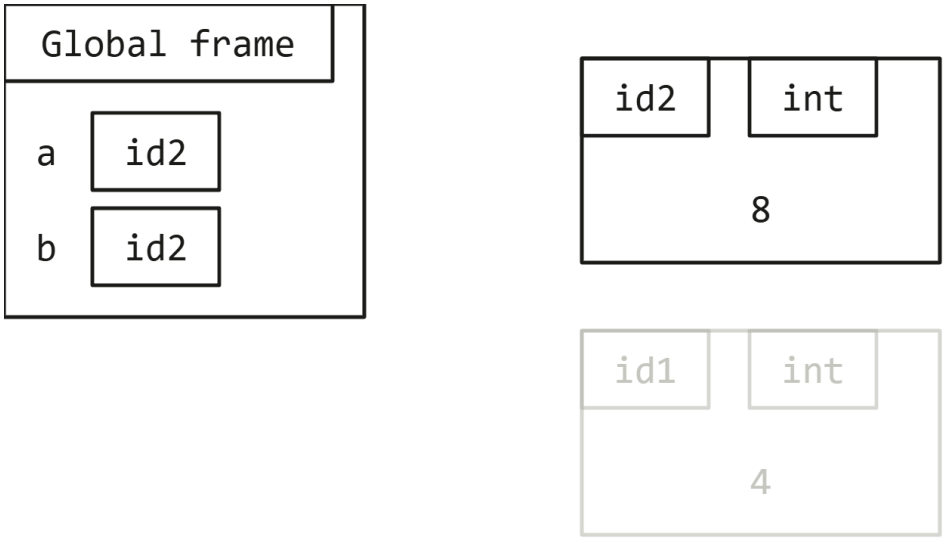
- ✓ 39
- B 22
- C 35
- D 20

swap_attempt.py

Program

```
 1  # We want to swap the values that a and b refer to: afterwards a should
 2  # refer to the value b used to refer to, and vice versa. The code must
 3  # work for ANY values of a and b.
 4  #
 5  # A generative AI tool produced the solution below. Starting from
 6  # a = 4 and b = 8, draw the memory model after it has run.
 7  a = 4
 8  b = 8
 9
10  # Possible solution
11  a = b
12  b = a
```

Memory model (step 4)


Global frame
a
id2
b
id2
id2
int
8
id1
int
4

After the AI's code has been executed, what values do `a` and `b` refer to?

```
1  a = 4
2  b = 8
3  a = b
4  b = a
```

A `a` refers to 4, `b` refers to 8

B `b` refers to 4, `a` refers to 8

C `a` and `b` both refer to 4

✓ `a` and `b` both refer to 8

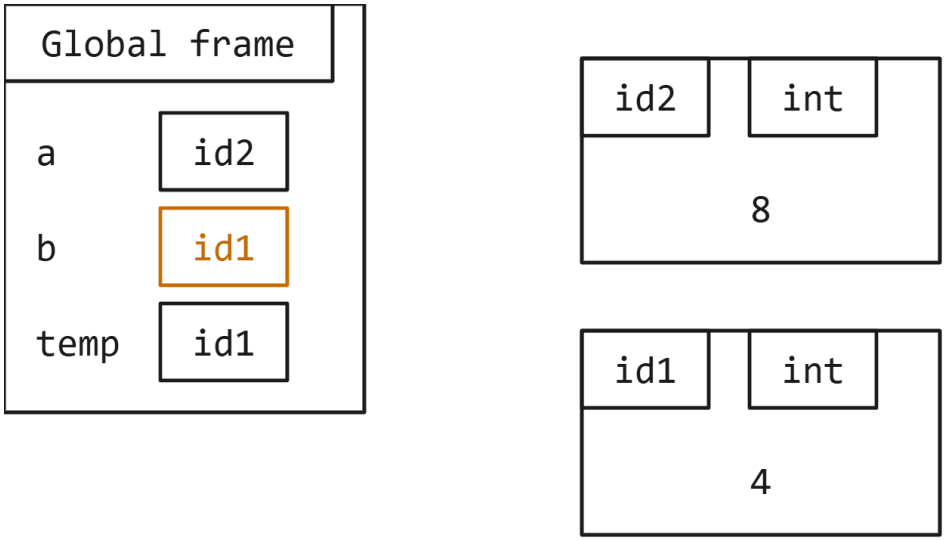
swap.py

Program

```
# Write a correct solution to swap the values of a and b, then draw
# the memory model diagram. Hint: use a third variable.
a = 4
b = 8

# Your solution
temp = a
a = b
b = temp
```

Memory model (step 5)


Global frame
a
id2
b
id1
temp
id1
id2
int
8
id1
int
4